

Statistical and Machine Learning

Neural Networks for Sequences (Teacher's Version)

Instructions

This document contains the discussion questions together with solutions, for use by the instructor.

Question 1: Architectures for Sequences

The lecture introduced three families of RNN architectures, distinguished by the mapping between input and output: **vec2seq**, **seq2vec**, and **seq2seq** (aligned and unaligned).

- (a) For each of the following tasks, identify which architecture is most appropriate, and briefly justify your answer:
 - i. Predicting the next word given the previous words (unconditional language generation).
 - ii. Classifying whether a movie review is positive or negative.
 - iii. Part-of-speech tagging (assigning a grammatical tag to every word in a sentence).
 - iv. Machine translation from English to French.
- (b) For the unaligned seq2seq (encoder-decoder) case, describe the role of the context vector $\mathbf{c}$, and explain why it can become a bottleneck for long input sequences.

Solution:

- (a)
 - i. **Vec2seq**: the model generates an output sequence $\mathbf{y}_{1:T}$ one token at a time from a hidden state, with $\mathbf{x} = \emptyset$ (unconditional language modelling).
 - ii. **Seq2vec**: a variable-length input sequence is mapped to a single fixed-length output (a class label), e.g. using the final hidden state $\mathbf{h}_T$.
 - iii. **Seq2seq, aligned case**: one label is predicted per input position, so $T' = T$ (dense sequence labelling).
 - iv. **Seq2seq, unaligned case**: input and output sequences generally have different lengths ($T' \neq T$), so an encoder-decoder architecture is required.
- (b) The context vector $\mathbf{c} = f_e(\mathbf{x}_{1:T})$ is a single fixed-length summary of the entire input sequence (e.g. the last state of the encoder RNN, or an average-pooled bidirectional RNN), from which the decoder generates the whole output sequence $\mathbf{y}_{1:T'} = f_d(\mathbf{c})$. Because the *entire* input must be compressed into this one vector regardless of its length, long sequences lead to information loss: the decoder has no direct access to individual input tokens, only to this compressed summary. This bottleneck motivates the use of attention, which lets the decoder access all encoder states directly rather than a single fixed vector.

Question 2: Training RNNs — Teacher Forcing and BPTT

- (a) Explain what is meant by **teacher forcing** when training a language model, and describe one problem it causes at test time. What is the standard fix, and how does it work?
- (b) Explain why computing the exact gradient via backpropagation through time (BPTT) requires $O(T^2)$ time for a sequence of length T , and how **truncated BPTT** addresses this.
- (c) Explain the vanishing/exploding gradient problem in RNNs. Why does it occur, and what are two ways of addressing it?

Solution:

- (a) Teacher forcing means that, during training, the input to the RNN at each step is the *ground-truth* previous label y_{t-1} , rather than the label sampled from the model itself. The problem this causes at test time is **exposure bias**: since the model has only ever conditioned on correct inputs during training, it may not know how to recover once its own generated output at test time deviates from the training distribution, leading to compounding errors. The standard fix is **scheduled sampling**: training begins with full teacher forcing, but at randomly chosen time steps the model's own sampled output is fed back in instead of the ground truth, with this fraction gradually increased over training.
- (b) The gradient $\partial \mathbf{h}_t / \partial \mathbf{w}_h$ at time t expands recursively into a sum over all previous time steps $i = 1, \dots, t-1$, each term involving a product of Jacobians from i up to t . Computing this sum for a single time step costs $O(t)$, and summing over all T time steps in the sequence gives $O(T^2)$ total cost. Truncated BPTT addresses this by only keeping the most recent K terms in the sum (i.e. only backpropagating through a window of K steps), reducing the cost to $O(TK)$.
- (c) Because the hidden state update multiplies by $\mathbf{W}_{hh}$ at every time step, and BPTT correspondingly multiplies Jacobians at every step going backwards in time, repeated multiplication can cause activations/gradients to shrink toward zero (vanish) or grow without bound (explode) as the sequence gets longer — depending on whether the relevant eigenvalues/spectral radius are below or above 1. Two ways to address this: (i) **gradient clipping**, a simple heuristic that rescales gradients once their norm exceeds a threshold (mainly addresses exploding gradients); (ii) controlling the **spectral radius** of $\mathbf{W}_{hh}$, e.g. by fixing it at random initialisation with spectral radius ≈ 1 (as in echo state networks), or more generally by using gated architectures (GRU/LSTM) that use additive state updates.

Question 3: Gating Mechanisms — GRU and LSTM

- (a) In a GRU, describe the roles of the reset gate $\mathbf{R}_t$ and the update gate $\mathbf{Z}_t$. What does it mean, in terms of model behaviour, if $\mathbf{R}_t \approx 0$? What does it mean if $\mathbf{Z}_t \approx 1$?
- (b) LSTMs augment the hidden state with a separate memory cell $\mathbf{C}_t$. Write down the update equation for $\mathbf{C}_t$, and explain how the forget and input gates allow the model to retain information over very long time spans.
- (c) Both GRUs and LSTMs rely on *additive* (rather than purely multiplicative) updates to their state. Explain why this design choice helps mitigate the vanishing gradient problem, drawing an analogy with another architectural idea seen earlier in the course.

Solution:

- (a) The reset gate $\mathbf{R}_t$ controls how much of the previous hidden state $\mathbf{H}_{t-1}$ contributes to the candidate state $\tilde{\mathbf{H}}_t$; if $\mathbf{R}_t \approx 0$, the old hidden state is discarded and the candidate state behaves like an MLP applied only to the current input $\mathbf{X}_t$, i.e. the model captures short-term information and effectively “forgets” the past. The update gate $\mathbf{Z}_t$ controls how the final state mixes the old state $\mathbf{H}_{t-1}$ and the candidate state $\tilde{\mathbf{H}}_t$; if $\mathbf{Z}_t \approx 1$, the old value $\mathbf{H}_{t-1}$ passes through essentially unchanged and the current input is ignored for that dimension, allowing the model to preserve information over many time steps (long-term memory).
- (b) $\mathbf{C}_t = \mathbf{F}_t \odot \mathbf{C}_{t-1} + \mathbf{I}_t \odot \tilde{\mathbf{C}}_t$. If the forget gate $\mathbf{F}_t = 1$ and the input gate $\mathbf{I}_t = 0$, the cell state carries its old value forward completely unchanged. This means information can, in principle, be preserved indefinitely across arbitrarily many time steps, since there is a direct additive (identity) path for the cell state rather than one requiring repeated nonlinear/multiplicative transformation.
- (c) An additive update means the gradient of the loss with respect to the state at an earlier time step can flow backward through a *sum* (with an identity component) rather than purely through a *product* of many Jacobians. This avoids gradients shrinking to zero (or exploding) as they are propagated through many time steps. This is directly analogous to the **residual/skip connections** discussed earlier in the course: adding the previous state (or the previous layer's input) directly to the output gives the

gradient a “highway” back through the network that does not require passing through a nonlinearity or a repeatedly-applied weight matrix at every step.

Question 4: 1D CNNs for Sequences

- Explain how a 1D convolution over a sequence produces a feature map, and how max-pooling over time is used to obtain a fixed-length representation for sequence classification.
- 1D CNNs can also be used to *generate* sequences, but must first be converted into **causal** convolutions. Explain what makes a convolution “causal”, and why this is necessary for generation but not for classification.
- Compare 1D CNNs to RNNs as an approach to sequence modelling: what is one practical advantage of CNNs over RNNs, and what kind of information can RNNs capture that a “vanilla” 1D CNN (with a fixed receptive field) cannot?

Solution:

- A 1D convolution slides a filter (kernel) of width k across the sequence: for each output channel c and position i , we compute $z_{ic} = \sum_d \mathbf{x}_{i-k:i+k,d}^\top \mathbf{w}_{d,c}$, i.e. an inner product between a local window of the input (summed across input channels d) and a learned filter, producing a $T \times C$ feature map. To reduce this to a single fixed-length vector $\mathbf{z} \in \mathbb{R}^C$ regardless of the (variable) sequence length T , we apply **max-pooling over time**: $z_c = \max_i z_{ic}$ for each channel c . This pooled vector is then passed to a softmax layer to obtain class probabilities.
- A convolution is **causal** if the output at position t only depends on past values ($y_{1:t-1}$, or inputs up to position $t - k$), with future positions masked out — unlike a standard convolution, which uses a window centred on (or otherwise including) positions after t . Causal convolutions are necessary for generation because a generative model must define a valid autoregressive factorisation $p(y_t | \mathbf{y}_{1:t-1})$: if the filter could see future tokens, the model would effectively be allowed to “cheat” by looking at values it is supposed to be predicting. This is not an issue for classification, since the entire input sequence is already fully observed at both training and test time — there is no future to leak information from.
- A practical advantage of 1D CNNs is that they are much easier and faster to train than RNNs, because they do not need to maintain a long, sequentially-updated hidden state — computation across positions can be parallelised, and there is no long chain of sequential dependencies to backpropagate through. What RNNs can capture that a vanilla 1D CNN cannot is (in principle) arbitrarily long-range dependencies: the RNN’s hidden state can, over many steps, accumulate information from the entire history of the sequence, whereas a 1D CNN with a fixed kernel size k can only directly combine information within its fixed receptive field (though this can be extended by stacking more layers or using dilated convolutions).

Question 5: The Attention Mechanism

- Define what is meant by queries, keys, and values in the context of attention, and write down the general formula for computing an attention output $\mathbf{z}_j$ as a function of $\mathbf{q}_j$ and the keys/values $(\mathbf{k}_i, \mathbf{v}_i)$.
- Scaled dot-product attention defines the score as $a(\mathbf{q}, \mathbf{k}) = \mathbf{q}^\top \mathbf{k} / \sqrt{d}$. Explain why we divide by $\sqrt{d}$, referring to the statistical properties of $\mathbf{q}$ and $\mathbf{k}$.
- Explain the connection between attention and Nadaraya-Watson kernel regression. In particular, what plays the role of the “query”, the “keys”, and the “values” in kernel regression?

Solution:

- Queries Q** describe what each output/input is “looking for”; **keys K** describe what each input “contains”, for the purposes of being matched against a query; **values V** describe what should actually be transmitted to the output once a match is found. The attention output is $\mathbf{z}_j = \sum_i \alpha_{ij} \mathbf{v}_i$, where the

weights α_{ij} (satisfying $0 \leq \alpha_{ij} \leq 1$ and $\sum_i \alpha_{ij} = 1$) are obtained by comparing the query $\mathbf{q}_j$ to each key $\mathbf{k}_i$ (e.g. via a softmax over similarity scores).

- (b) If the entries of $\mathbf{q}$ and $\mathbf{k}$ are independent, zero-mean, unit-variance random variables, then their dot product $\mathbf{q}^\top \mathbf{k}$ has mean 0 but variance d (growing linearly with the dimension d). Dividing by $\sqrt{d}$ keeps the variance of the score at 1 regardless of d . Without this scaling, as d grows large the dot products would tend to have very large magnitude, pushing the softmax into a very “peaked”/saturated regime with extremely small gradients, making training unstable.
- (c) In kernel regression, the **query** is the test point $\mathbf{x}$, the **keys** are the training inputs $\mathbf{x}_i$, and the **values** are the training labels y_i . The attention weight $\alpha_i(\mathbf{x}, \mathbf{x}_{1:n})$ is exactly the normalised kernel similarity (e.g. Gaussian kernel) between the test point and each training input, and the prediction $f(\mathbf{x}) = \sum_i \alpha_i(\mathbf{x}, \mathbf{x}_{1:n}) y_i$ is precisely of the same form as the attention formula in Part (a), making attention a parametric, learned generalisation of this classical non-parametric idea.

Question 6: Transformers and Language Models

- (a) Self-attention allows all outputs to be computed in parallel during training, unlike RNNs. Explain why this is possible, and why an RNN cannot be parallelised in the same way.
- (b) Since self-attention is permutation invariant, transformers use **positional encodings**. Explain what problem this solves, and describe one desirable property of the sinusoidal positional encoding scheme discussed in the lecture.
- (c) Compare BERT and GPT as approaches to language modelling:
 - i. Which model is causal (autoregressive), and which is non-causal?
 - ii. What pretraining objective does each use?
 - iii. Why can GPT be used directly to generate text, but BERT cannot?

Solution:

- (a) In self-attention, each output $\mathbf{y}_i = \text{Attn}(\mathbf{x}_i, (\mathbf{x}_1, \mathbf{x}_1), \dots, (\mathbf{x}_n, \mathbf{x}_n))$ is computed directly from the (already available) full set of inputs, and does not depend on any other output having been computed first. At training time, since all target tokens are already known (e.g. via teacher forcing), every output position can therefore be computed simultaneously. An RNN, by contrast, computes its hidden state $\mathbf{h}_t$ recursively from $\mathbf{h}_{t-1}$, so each step depends sequentially on the previous one and cannot be computed until the earlier steps are finished — this is an inherently sequential bottleneck.
- (b) Attention treats its inputs as an unordered set (permutation invariant), so without additional information the model cannot tell word order apart — e.g. it would process “the cat sat” and “sat the cat” identically. Positional encodings solve this by adding a position-dependent vector $\mathbf{P}$ to the word embeddings $(\mathbf{X} + \mathbf{P})$, giving the model access to ordering information. One desirable property of the sinusoidal scheme is that the encoding of one position is linearly predictable from that of another, given their relative offset: $\mathbf{p}_{t+\phi} = f(\mathbf{p}_t)$ for a linear (rotation) transformation f . Since this is a rotation, for small ϕ we have $\mathbf{p}_{t+\phi} \approx \mathbf{p}_t$, giving the model a useful inductive bias about relative position, and it can also be computed for sequences of arbitrary length (up to some maximum C), unlike a learned embedding table.
- (c)
 - i. **GPT** is causal/autoregressive; **BERT** is non-causal.
 - ii. GPT is trained with a standard **language modelling** objective: predict the next token, $p(x_t | \mathbf{x}_{1:t-1})$. BERT is trained with a **masked / cloze (fill-in-the-blank)** objective: given a randomly masked version of the input, predict the missing tokens, minimising the negative log pseudo-likelihood $\mathcal{L} = \mathbb{E}_{\mathbf{x}} \mathbb{E}_{\mathbf{m}} \sum_{i \in \mathbf{m}} -\log p(x_i | \mathbf{x}_{-\mathbf{m}})$ (BERT is also trained on a next-sentence-prediction task).